



Universitat
de les Illes Balears

TREBALL DE FI DE GRAU

APRENENTATGE PER REFORÇ PROFUND PER AL JOC DEL TRUC

Josep Ferriol Font

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Any acadèmic 2025-26

APRENTATGE PER REFORÇ PROFUND PER AL JOC DEL TRUC

Josep Ferriol Font

Treball de Fi de Grau

Escola Politècnica Superior

Universitat de les Illes Balears

Any acadèmic 2025-26

Paraules clau del treball: aprenentatge per reforç, truc, jocs de cartes, informació parcial, curriculum learning, self-play

Tutor: Dr. Francesc Xavier Gayà Morey

Cotutor: Dr. Gabriel Moyà Alcover

Segons estudis demogràfics, una gran part de la població mundial neix en contextos on l'accés a l'educació i la sanitat no estan garantits. Tenir la sort de néixer a Europa, en un estat de dret que reconeix la salut i l'educació com a pilars fonamentals de la societat, és un privilegi que no hauria de donar-se per descomptat. Per això, vull agrair haver tingut l'oportunitat de cursar una educació pública i de qualitat a la meua terra,
Mallorca.

A més, un agraïment especial dirigit a la meua família, els altres grans patrocinadors d'aquesta carrera. El seu suport incondicional ha estat el fonament sobre el qual s'ha construït tot el que he aconseguit.

SUMARI

Sumari	iii
Índex de figures	v
Índex de taules	vii
Llista d'acrònims	ix
Resum	xi
1 Introducció	1
1.1 La intel·ligència artificial en el joc	2
1.2 El repte de la informació parcial	2
1.3 El Truc: engany, parella i farol	3
1.4 Objectius i estructura d'aquest treball	3
2 Fonaments Teòrics	5
2.1 Aprenentatge per Reforç	5
2.2 Estat de l'art en jocs d'informació parcial	8
2.3 Deep Q-Network	9
2.4 Proximal Policy Optimization	9
2.5 Curriculum Learning	10
2.6 Deep Recurrent Q-Learning	11
2.7 Self-play i equilibri de Nash	11
2.8 Neural Fictitious Self-Play	12
3 El Joc del Truc	15
3.1 Regles del joc	15
3.2 Arquitectura del simulador	17
3.3 El motor de joc	19
4 Conclusions	21
A Hiperparàmetres dels models	23
B Resultats complets per fase	25
Bibliografia	27

ÍNDIX DE FIGURES

2.1	Cicle d'interacció agent–entorn en AR.	6
2.2	Model gràfic del POMDP: els estats s_t (cercles continus) són ocults; l'agent només observa o_t (cercles discontinus) generats per la funció d'emissió O	8
2.3	Arquitectura DQN: la xarxa objectiu θ^- (discontínua) proporciona objectius de Bellman estables; l'experience replay $\mathcal{D}$ trenca la correlació temporal entre mostres.	9
2.4	Arquitectura Actor-Critic de PPO: extractor de característiques compartit amb dos caps independents, un per a la política i un per a la funció de valor.	10
2.5	Currículum de dues etapes: mans individuals amb recompensa densa, seguides de partides senceres amb transferència dels pesos apresos.	11
2.6	NFSP: el bucle RL genera accions que s'acumulen al reservoir buffer; el bucle SL aprèn per imitació la política promig $\bar{\pi}$, que s'usa com a oponent durant l'entrenament.	13
3.1	Arquitectura MVC: el controlador només coneix les interfícies.	17
3.2	Fases d'un torn. Si les senyes estan desactivades, la Fase 0 se salta.	19

ÍNDIX DE TAULES

2.1	Components de la tupla MDP.	6
2.2	Comparativa dels algorismes d'AR emprats al treball.	13
3.1	Jerarquia de força de les cartes, de major a menor.	16
3.2	Escala d'apostes de l'Envit.	16
3.3	Escala d'apostes del Truc.	17
3.4	Mètodes principals del contracte Model.	18
3.5	Mètodes principals del contracte Vista.	18

LLISTA D'ACRÒNIMS

IA	Intel·ligència Artificial
AR	Aprenentatge per Reforç
DRL	Deep Reinforcement Learning
DQN	Deep Q-Network
PPO	Proximal Policy Optimization
NFSP	Neural Fictitious Self-Play
MDP	Markov Decision Process
POMDP	Partially Observable Markov Decision Process
MVC	Model-Vista-Controlador
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
DRQN	Deep Recurrent Q-Network
TFG	Treball Final de Grau

RESUM

[Resum de 250–500 paraules: problema, mètode, resultats, conclusions.]

Paraules clau: *reinforcement learning, truc, jocs de cartes, DQN, PPO, NFSP, informació parcial, curriculum learning, self-play.*

INTRODUCCIÓ

El joc és una de les activitats més antigues de la humanitat. Més que un objecte d'estudi, és una pràctica social: un espai on les persones s'enfronten, es coordinen, negocien i, sovint, s'ensenyen mútuament. Els antropòlegs el descriuen com un mecanisme de transmissió cultural i d'entrenament de la intel·ligència, present en totes les societats conegudes [1]. Jugar no és només passar l'estona: és practicar la presa de decisions sota incertesa, gestionar el risc i llegir les intencions dels altres.

Els jocs tradicionals hi afegeixen una capa de patrimoni i cultura. Aquests no són simples entreteniments: són codis culturals transmesos de generació en generació, amb vocabulari propi, rituals i regles que reflecteixen el pensament social de cada territori. El problema és que necessiten jugadors que coincideixin al mateix lloc i al mateix moment. La dispersió geogràfica, l'envelliment dels cercles de jugadors i el canvi d'hàbits d'oci fan que sigui cada cop més difícil. Sense adversaris ni parella, el joc s'extingeix per inanició, no per desinterès.

No record exactament quan vaig aprendre a jugar a cartes. Als tretze o catorze anys ja hi jugaven tots els meus amics del poble. El que sé és que he tengut la sort de poder comptar amb els padrins, ties i altre gent, que m'han pogut ensenyar els distints jocs tradicionals, entre ells jocs de cartes. Al meu poble, jocs com el Terceti (que avui quasi ja no es juga a cap altre lloc) o el Truc formen part de la vida quotidiana, transmesos de generació en generació sense que ningú ho pensés gaire.

Però amb els anys m'he anat adonant que cada cop és més difícil trobar gent per seure a una taula. El cercle de jugadors envelleix, i quan no hi ha prou persones disponibles el Truc simplement no es pot jugar. Aquesta inquietud és el que m'ha duit a fer aquest Treball Final de Grau (TFG): crear una versió del Truc que es pugui jugar contra un agent d'aprenentatge per reforç, de manera que el joc pugui continuar viu encara que no hi hagi quatre humans disponibles.

1.1 La intel·ligència artificial en el joc

Darrerament, la relació entre jocs i Intel·ligència Artificial (IA) ha anat més lluny de la preservació cultural. Des de fa dècades, els jocs han servit també com a banc de prova per als investigadors, ja que, en ser entorns tancats amb regles precises i un criteri d'èxit mesurable, permeten avaluar algorismes de manera objectiva i reproducible. L'any 1997, Deep Blue va derrotar el campió mundial d'escacs Garry Kasparov, una fita que semblava inalcançable una dècada abans [2]. El 2016, AlphaGo va superar Lee Sedol al joc del Go, un problema considerat exponencialment més complex pels seus 10^{170} estats possibles [3]. El 2019, l'agent OpenAI Five va derrotar equips professionals de Dota 2 en partides de cinc jugadors per banda [4].

El denominador comú d'aquests avenços és el Deep Reinforcement Learning (DRL): la combinació de l'Aprenentatge per Reforç (AR) clàssic amb xarxes neuronals profundes. L'AR permet a un agent aprendre interactuant amb un entorn i rebent recompenses; les xarxes profundes li permeten representar espais d'estats molt complexos sense necessitat de definir manualment les característiques rellevants. La combinació dels dos ha permès que agents sense cap coneixement inicial arribin a nivells humans en diferents dominis (escacs, Go, Dota 2, etc.). Aquest treball segueix el mateix enfocament: xarxes neuronals profundes combinades amb AR per aprendre a jugar al Truc partint de zero.

1.2 El repte de la informació parcial

Tots aquests èxits, però, es produeixen en jocs d'*informació completa*: tots els jugadors veuen l'estat sencer del joc en tot moment. Els escacs, el Go i els jocs de taulell en general pertanyen a aquesta categoria. En canvi, molts dels jocs que els humans troben més fascinants, i estratègicament rics, amaguen informació: les cartes dels adversaris, les intencions futures, els recursos ocults.

Formalment, un joc d'informació parcial es modela com un Partially Observable Markov Decision Process (POMDP): l'agent rep observacions parcials de l'estat real del món i ha d'actuar sense visibilitat completa. L'estratègia òptima ja no consisteix a calcular la millor jugada donada la posició, sinó a raonar sobre els estats ocults i el possible comportament d'un adversari.

Els jocs de pòquer han estat el principal camp d'experimentació en informació parcial. El 2017, Libratus va guanyar jugadors professionals de Texas Hold'em de dos jugadors (*heads-up*) [5]. El 2019, Pluribus va estendre aquest resultat al pòquer de sis jugadors, un entorn multiagent molt més complex on les tècniques clàssiques de teoria de jocs no escalen directament [6]. Ambdós agents aproximen un *equilibri de Nash*: una estratègia tal que cap jugador pot millorar el seu resultat canviant unilateralment de política.

Malgrat aquests progressos, el camp és lluny d'estar tancat. Cada joc d'informació parcial té la seva pròpia estructura d'observació, el seu espai d'accions i les seves dinàmiques estratègiques. Un agent entrenat al pòquer no es pot transferir directament a un altre joc de cartes: cal dissenyar l'entorn, definir l'espai d'estats i triar els algorismes adequats a cada cas.

1.3 El Truc: engany, parella i farol

El Truc és un joc de cartes tradicional d'arrel catalana i valenciana, arrelat a les Illes Balears i al País Valencià [7]. Es juga per parelles (dos contra dos) amb una baralla espanyola de 36 cartes, i guanya el primer equip que arriba als 24 punts d'una *cama*. Cada mà es disputa en dues apostes independents: l'**Envit** (sobre la força de la mà en punts) i el **Truc** (sobre qui guanya les rondes de carta alta). Les regles completes, la jerarquia de cartes i el model formal es detallen al capítol 3.

La mecànica no és el que fa del Truc un repte per a la IA. És el **farol**: qui té cartes dolentes pot cantar Truc amb convicció i guanyar el punt sense ensenyar-les. A més, les parelles es coordinen mitjançant *senyes*, un sistema de gestos facials codificats que els adversaris poden intentar llegir i explotar. Com es diu al joc: “*El Truc no és només matemàtiques i memòria. És un joc de cares, de silencis i de mentides.*”

Tot plegat, el Truc és un POMDP cooperatiu-competitiu: l'agent ha de gestionar informació oculta, coordinar-se sense comunicació verbal explícita i decidir a cada torn si enganyar o ser sincer sobre la força de la seva mà. Aquesta combinació de factors el fa substancialment diferent dels jocs estudiats fins ara en la literatura.

1.4 Objectius i estructura d'aquest treball

Per complir amb l'objectiu d'aquest TFG (*desenvolupar i avaluar agents d'AR capaços de jugar al Truc de manera competitiva, partint de zero i sense cap coneixement previ del joc*) ens cal primer implementar una versió digital completa i jugable del Truc (incloent les modalitats de 2 vs 2 amb senyes). L'entrenament dels agents es circumscriu a la variant d'1 vs 1 sense senyes; l'extensió a agents cooperatius per parelles s'identifica com a línia de recerca futura. Concretament, el treball s'articula al voltant de tres objectius principals:

1. **Modelització, implementació i aplicació jugable.** Implementar el motor complet del joc (envit, truc, apostes i senyes) seguint una arquitectura Model-Vista-Controlador (MVC) que desacobla nítidament la lògica del joc, el control i la interfície. L'aplicació compta amb dues interfícies: consola (text) i escriptori (gràfica amb imatges de naips i interacció per ratolí). La interfície d'escriptori permet partides 1 vs 1 (humà contra humà o humà contra agent) i 2 vs 2 humà contra humà amb suport de senyes. El motor s'exposa alhora com a entorn estàndard compatible amb les biblioteques d'AR (RLCard [8] i Stable-Baselines3 [9]).
2. **Comparació d'algorismes i tècniques d'entrenament.** Avaluar quatre algorismes (Deep Q-Network (DQN) i Neural Fictitious Self-Play (NFSP) via RLCard, i DQN i Proximal Policy Optimization (PPO) via SB3) en condicions homogènies, i estudiar l'efecte del *curriculum learning* (entrenar primer en mans individuals i després en partides senceres), de l'ús d'extractors de característiques convolucionals, de la memòria recurrent (GRU) i del *self-play* sobre la convergència i el rendiment dels agents.
3. **Avaluació.** Mesurar el rendiment de cada agent contra un agent de regles heurístiques i analitzar si els agents aprenen comportaments estratègics (gestió de l'envit, ús del farol al truc) o es limiten a polítiques reactives.

1. INTRODUCCIÓ

El treball s'organitza de la manera següent. El capítol 2 presenta el marc teòric: els fonaments de l'AR, els algorismes emprats i l'estat de l'art en jocs d'informació parcial. El capítol 3 descriu la lògica del joc i l'arquitectura del simulador. El capítol ?? detalla el disseny dels entorns d'AR. El capítol ?? explica la metodologia d'aprenentatge per reforç i les fases d'entrenament. El capítol ?? presenta i discuteix els resultats experimentals. Finalment, el capítol ?? recull les conclusions i les línies de treball futur.

FONAMENTS TEÒRICS

Aquest capítol presenta els fonaments teòrics que sustenten el treball: els conceptes bàsics de l'AR, els algorismes emprats i l'estat de l'art en jocs d'informació parcial. Cada secció s'orienta específicament als components que apareixeran a la metodologia d'entrenament.

2.1 Aprenentatge per Reforç

L'AR és el paradigma d'aprenentatge automàtic en el qual un agent aprèn a actuar en un entorn mitjançant la interacció directa, guiat per senyals de recompensa [10]. A diferència de l'aprenentatge supervisat, no existeix cap conjunt d'exemples etiquetats ni un tercer que indiqui la resposta correcta; a diferència de l'aprenentatge no supervisat, sí que hi ha una senyal d'avaluació (la recompensa) que orienta l'aprenentatge.

Aquesta secció presenta els fonaments formals de l'AR: la formalització com a Markov Decision Process (MDP), les funcions de valor i les equacions de Bellman, el problema d'exploració, la distinció entre algorismes *on-policy* i *off-policy*, i l'extensió al cas d'observació parcial (POMDP).

Procés de Decisió de Markov

L'entorn es formalitza com un Markov Decision Process (MDP), definit per la tupla (S, A, P, R, γ) on cada component té el significat que recull la Taula 2.1.

Component	Descripció
S	Conjunt d'estats de l'entorn
A	Conjunt d'accions disponibles per a l'agent
$P(s' s, a)$	Probabilitat de transitar a s' prenent l'acció a des de s
$R(s, a, s')$	Recompensa immediata esperada en la transició
$\gamma \in [0, 1)$	Factor de descompte (recompenses futures vs. immediates)

Taula 2.1: Components de la tupla MDP.

Una *política* π és la regla de decisió de l'agent: en la forma estocàstica, $\pi(a | s)$ expressa la probabilitat de seleccionar l'acció a quan l'estat és s ; en la forma determinista, $\pi : S \rightarrow A$ assigna directament una acció a cada estat. El cicle bàsic d'interacció és el representat a la Figura 2.1: a cada pas de temps t , l'agent observa s_t , tria $a_t \sim \pi(\cdot | s_t)$, rep la recompensa r_t i l'entorn transita a $s_{t+1} \sim P(\cdot | s_t, a_t)$.

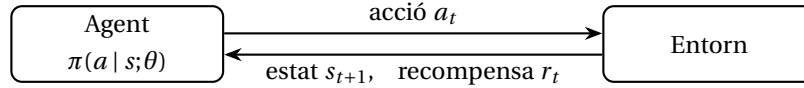


Figura 2.1: Cicle d'interacció agent-entorn en AR.

L'objectiu de l'agent és trobar la política òptima π^* que maximitzi el *retorn acumulat descomptat*:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

El factor γ controla l'horitzó temporal efectiu: quan $\gamma \rightarrow 0$ l'agent és miop i prioritza les recompenses immediates; quan $\gamma \rightarrow 1$ valora el futur quasi tant com el present.

Funcions de valor i equacions de Bellman

Per saber si una política π és bona, cal quantificar el retorn esperat que l'agent obtindrà seguint-la. Per fer-ho s'utilitzen dues funcions de valor complementàries: la *funció de valor d'estat* $V^\pi(s)$ i la *funció de valor d'acció* $Q^\pi(s, a)$:

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a].$$

Aquestes funcions satisfan les *equacions de Bellman*, que les expressen de forma recursiva a partir del pas immediatament següent:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')], \quad (2.1)$$

$$Q^\pi(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a')]. \quad (2.2)$$

Quan la política és l'òptima π^* , les equacions es simplifiquen substituint la suma sobre accions per un màxim:

$$V^*(s) = \max_a \mathbb{E}[r + \gamma V^*(s') | s, a], \quad (2.3)$$

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a') | s, a]. \quad (2.4)$$

La relació entre totes dues és $V^*(s) = \max_a Q^*(s, a)$, i la política òptima s'obté directament com $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$. Quan l'espai d'estats és massa gran per representar Q^* en una taula, s'aproxima amb una xarxa neuronal $Q(s, a; \theta)$, donant lloc a la família d'algorismes de DRL.

Exploració i explotació

Fins i tot amb les equacions de Bellman ben definides, aprendre Q^* per interacció directa planteja un dilema pràctic: l'agent ha de decidir si aplica l'acció que creu millor (**explotació**) o en prova una de nova per obtenir més informació (**exploració**). Sense exploració suficient, l'agent queda atrapat en polítiques subòptimes; amb massa exploració, no convergeix.

L'estratègia ϵ -greedy és la solució més senzilla: amb probabilitat $1 - \epsilon$ tria l'acció de valor màxim i amb probabilitat ϵ n'escull una uniformement a l'atzar. Durant l'entrenament, ϵ decau des d'un valor proper a 1 fins a un valor residual baix. Els algorismes de gradient de política com PPO adopten un enfocament diferent: afegeixen un terme d'entropia $\mathcal{H}[\pi]$ a la funció objectiu, penalitzant les polítiques massa deterministes i incentivant la diversitat d'accions de forma contínua.

Polítiques *on-policy* i *off-policy*

Els algorismes *on-policy* (com PPO) aprenen únicament de dades generades per la política actual; els *off-policy* (com DQN) poden reutilitzar transicions passades emmagatzemades en un *replay buffer*, cosa que millora l'eficiència de mostres però requereix correcció de la distribució.

Observació parcial: el model POMDP

Els models MDP vistos fins ara assumeixen que l'agent observa l'estat complet del sistema en cada instant. En molts entorns reals, i en particular al Truc, aquesta suposició no es compleix: els adversaris amaguen les seves cartes i les intencions futures romanen ocultes.

Un Partially Observable Markov Decision Process (POMDP) estén el MDP amb dues components addicionals (un conjunt d'observacions Ω i una funció d'emissió O), donant la tupla completa $(S, A, P, R, \gamma, \Omega, O)$. La funció $O(o | s', a)$ especifica la probabilitat de rebre l'observació $o \in \Omega$ després de transitar a l'estat s' prenent l'acció a . L'agent no veu s_t directament, sinó una observació parcial o_t que en conté menys informació, tal com il·lustra la Figura 2.2.

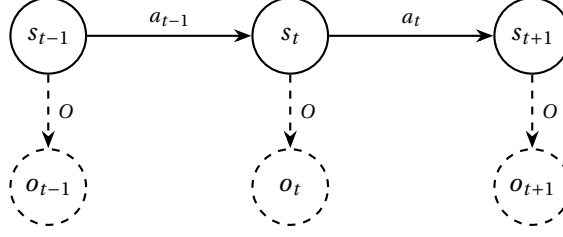


Figura 2.2: Model gràfic del POMDP: els estats s_t (cercles continus) són ocults; l'agent només observa o_t (cercles discontinus) generats per la funció d'emissió O .

La política òptima en un POMDP ha de raonar sobre la *distribució de creença* (*belief state*) $b_t(s) = P(s_t = s \mid o_{1:t}, a_{1:t-1})$. Quan es rep una nova observació o_{t+1} , la creença s'actualitza aplicant el teorema de Bayes:

$$b_{t+1}(s') \propto O(o_{t+1} \mid s', a_t) \sum_{s \in S} P(s' \mid s, a_t) b_t(s).$$

En la pràctica, mantenir b_t explícitament és intractable per a espais d'estats grans. En el seu lloc s'utilitzen xarxes recurrents (LSTM, GRU) que comprimeixen l'historial d'observacions en un vector d'estat ocult, actuant com a representació implícita de la creença.

L'entorn del Truc és precisament un POMDP: l'agent rep una observació que inclou la seva mà i l'historial d'apostes visible, però no les cartes dels adversaris.

2.2 Estat de l'art en jocs d'informació parcial

El pòquer ha estat el banc de prova principal per a la IA en jocs d'informació parcial, i els sistemes basats en Minimització del Penediment Contra-factual (*Counterfactual Regret Minimization*, CFR) han assolit resultats sobrehumans.

Libratus [5] (Brown & Sandholm, 2017) guanya jugadors professionals de Texas Hold'em de dos jugadors. CFR minimitza iterativament el penediment contra-factual (la millora hipotètica si s'hagués pres una acció diferent) i convergeix a l'equilibri de Nash. Libratus resol sub-jocs en temps real per refinar la seva estratègia durant la partida.

Pluribus [6] (Brown & Sandholm, 2019) estén aquest resultat al pòquer de sis jugadors combinant Monte Carlo CFR, una estratègia de base (*blueprint*) obtinguda per self-play i la resolució de sub-jocs en temps real. És la primera IA a superar humans en un entorn multi-jugador d'informació parcial.

Limitacions dels enfocaments CFR

Malgrat els seus èxits, CFR presenta tres limitacions que motiven l'ús de DRL en aquest treball. El cost creix exponencialment amb la profunditat de l'arbre i el nombre de jugadors, cosa que exigeix abstraccions manuals específiques per a cada joc. A més, aquestes abstraccions no es transfereixen: redissenyar-les per al Truc (amb apostes menys estructurades i senyes) seria tan costós com el problema original. Finalment, CFR no aprèn representacions compartides, de manera que no es beneficia de la generalització de les xarxes neuronals.

L'enfocament DRL (DQN, PPO, NFSP) evita modelar l'arbre explícitament i generalitza a nous jocs sense redissenyar abstraccions, a canvi d'una convergència a Nash menys garantida teòricament.

2.3 Deep Q-Network

Deep Q-Network (DQN) [11] fou el primer algoritme de DRL a demostrar aprenentatge de nivell humà en entorns d'alta dimensionalitat (jocs Atari), combinant Q-learning amb xarxes neuronals profundes.

En lloc d'una taula $Q(s, a)$, DQN aproxima la funció de valor d'acció amb una xarxa neuronal $Q(s, a; \theta)$ parametritzada per θ . Aprèn minimitzant la pèrdua:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right].$$

Dues innovacions clau estableixen l'entrenament:

- **Experience Replay:** les transicions (s, a, r, s') s'emmagatzemen en un buffer $\mathcal{D}$ i es mostregen aleatòriament. Això trenca la correlació temporal entre mostres consecutives, requisit per a la convergència de la xarxa.
- **Xarxa objectiu θ^- :** una còpia de θ que s'actualitza periòdicament (cada N passes) i s'usa per calcular els objectius de Bellman. Evita l'oscil·lació causada per actualitzar simultàniament la xarxa d'estimació i l'objectiu.

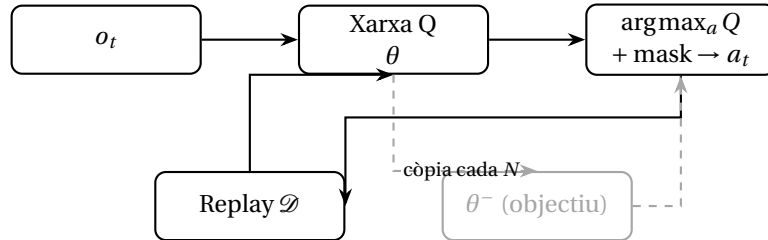


Figura 2.3: Arquitectura DQN: la xarxa objectiu θ^- (discontínua) proporciona objectius de Bellman estables; l'experience replay $\mathcal{D}$ trenca la correlació temporal entre mostres.

2.4 Proximal Policy Optimization

Proximal Policy Optimization (PPO) [12] és un algoritme Actor-Critic on-policy que s'ha convertit en l'estàndard pràctic de l'AR gràcies a la seva estabilitat i eficiència computacional.

Actor-Critic

PPO manté dues xarxes: l'*actor* $\pi(a | s; \theta)$ que aprèn la política, i el *crític* $V(s; \phi)$ que estima la funció de valor. L'actor s'entrena per maximitzar el retorn esperat; el crític proporciona una línia base que redueix la variança del gradient.

L'avantatge estimat $\hat{A}_t$ mesura si l'acció presa va ser millor o pitjor del previst pel crític, i s'estima mitjançant el mètode GAE (*Generalized Advantage Estimation*).

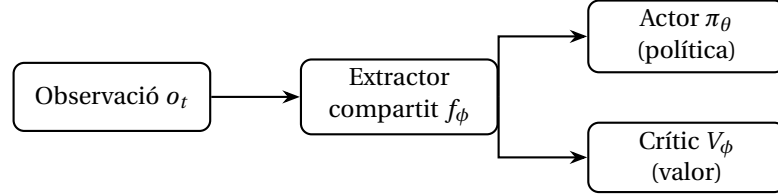


Figura 2.4: Arquitectura Actor-Critic de PPO: extractor de característiques compartit amb dos caps independents, un per a la política i un per a la funció de valor.

Objectiu amb retall

La innovació central de PPO és l'objectiu amb retall (*clipped surrogate objective*), que limita l'actualització de la política per evitar passos massa grans:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)],$$

on $r_t(\theta) = \pi(a_t | s_t; \theta) / \pi(a_t | s_t; \theta_{\text{old}})$ és el ràtio entre la política nova i l'antiga, i $\epsilon = 0,2$.

La pèrdua total afegeix un terme de valor $\mathcal{L}^V$ i un bonus d'entropia $\mathcal{H}[\pi]$ per mantenir l'exploració:

$$\mathcal{L} = \mathcal{L}^{\text{CLIP}} - c_1 \mathcal{L}^V + c_2 \mathcal{H}[\pi].$$

Paral·lelisme

Com que PPO és *on-policy*, necessita generar mostres noves en cada iteració. Per compensar aquest cost, l'algorisme aprofita el paral·lelisme de manera natural: múltiples còpies de l'entorn s'executen simultàniament i alimenten un únic agent central, tal com proposen Schulman et al. [12]. Aquesta arquitectura permet escalar el nombre de passos per unitat de temps de forma gairebé lineal amb el nombre d'entorns paral·lels, compensant l'eficiència de mostres inferior respecte als mètodes *off-policy*.

2.5 Curriculum Learning

El *curriculum learning* [13] és una estratègia d'entrenament que presenta els exemples o les tasques en ordre creixent de dificultat, imitant la manera com els humans aprenen.

En el context de l'AR, el problema que motiva l'ús del curriculum és el de la **recompensa escassa**: en una partida sencera de Truc, l'episodi pot durar fins a ~ 150 passos i el senyal de recompensa rellevant arriba tard i de forma difusa. Amb un horitzó tan llarg, l'agent té dificultats per atribuir les recompenses a les decisions que les van causar.

Aplicació al Truc

S'ha dissenyat un currículum de dues etapes:

1. **Etapla de mans** (12M passos): l'agent s'entrena en TrucGameMa, on cada mà és un episodi independent de ~ 15 passos amb recompensa densa normalitzada pts/24. La recompensa és proporcional als punts guanyats a cada mà.
2. **Etapla de partides** (12M passos addicionals): els pesos apresos es transfereixen a TrucGame, on l'episodi és una partida sencera fins a 24 punts. L'agent parteix d'una política ja formada i l'afinament es produeix sobre el comportament estratègic a llarg termini.

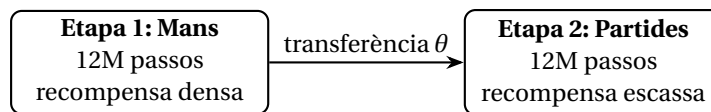


Figura 2.5: Currículum de dues etapes: mans individuals amb recompensa densa, seguides de partides senceres amb transferència dels pesos apresos.

El risc principal és l'**oblit catastròfic**: els algorismes on-policy com PPO descarten les dades antigues, cosa que pot fer que l'agent obliidi el que va aprendre a l'etapa de mans quan s'exposa a les partides senceres.

2.6 Deep Recurrent Q-Learning

Deep Q-Network (DQN) assumeix que l'observació o_t és una representació suficient de l'estat per prendre decisions òptimes. En un POMDP això no és cert: sense memòria de les observacions passades, l'agent pot prendre decisions subòptimes quan l'estat ocult és rellevant.

Deep Recurrent Q-Network (DRQN) [14] proposa substituir la primera capa densa de DQN per una Gated Recurrent Unit (GRU) o LSTM que manté un estat ocult (h_t, c_t) propagat a través dels passos de temps. Així, l'aproximació de la funció de valor depèn no sols de o_t sinó de tota la seqüència d'observacions passades, el que equival a aproximar la funció de valor sobre la *creença* (distribució de probabilitat sobre estats ocults).

Entrenament seqüencial

L'entrenament de DRQN propaga els gradients a través del temps (BPTT, *Back-propagation Through Time*) sobre segments de trajectòria de longitud L . La pèrdua s'acumula als últims L passos de la seqüència per evitar que els gradients s'extingueixin en contextos molt llargs.

2.7 Self-play i equilibri de Nash

En un joc de suma zero entre dos jugadors, un **equilibri de Nash** és un parell de polítiques (π^*, σ^*) tal que cap dels dos jugadors pot incrementar el seu retorn esperat canviant unilateralment la seva política:

$$V(\pi^*, \sigma^*) \geq V(\pi, \sigma^*) \quad \forall \pi, \quad V(\pi^*, \sigma^*) \leq V(\pi^*, \sigma) \quad \forall \sigma.$$

Una política en equilibri de Nash és *no explorable*: cap estratègia adversària pot millorar contra ella.

Fictitious Play i self-play modern

El *Fictitious Play* (Brown, 1951) és un algorisme iteratiu on cada jugador best-responds a la política promig de l'adversari. Convergeix a l'equilibri de Nash en jocs de suma zero de dos jugadors.

El self-play modern [6] opera de manera similar: l'agent s'entrena jugant contra còpies antigues de si mateix (pool de snapshots), la qual cosa evita l'especialització excessiva contra un únic oponent i afavoreix la convergència cap a polítiques robustes.

Mètriques de robustesa

Per mesurar la qualitat de la política més enllà del win rate brut, s'han definit dues mètriques:

- $\text{metric_robust} = \overline{WR} - \lambda \sigma(WR_{\text{variant}})$, que penalitza la variança del win rate entre variants d'oponent (amb $\lambda = 0,5$).
- $\text{exploit_selfplay} = |WR_{\text{vs snapshots}} - 50|$, que mesura la distància a l'equilibri de Nash: 0 indica que cap dels dos jugadors pot guanyar sistemàticament a l'altre.

2.8 Neural Fictitious Self-Play

Neural Fictitious Self-Play (NFSP) [15] és un marc d'entrenament per a jocs d'informació imperfecta que combina l'AR per aprendre la millor resposta (*best response*) amb aprenentatge supervisat per aprendre la política promig, aproximant així l'equilibri de Nash sense resoldre explícitament l'arbre de joc.

Dos bucles en paral·lel

1. **Bucle RL (millor resposta)**: un agent PPO maximitza la recompensa jugant contra el pool d'oponents (que inclou còpies antigues i la política promig). Aquest agent aprèn una política que *best-responds* a les polítiques de l'històric.
2. **Bucle SL (política promig)**: una xarxa auxiliar AveragePolicyNet s'entrena per cross-entropy per imitar les accions preses pel bucle RL al llarg de tot l'entrenament. Rep com a etiquetes les accions del bucle RL i aprèn la distribució promig d'accions.

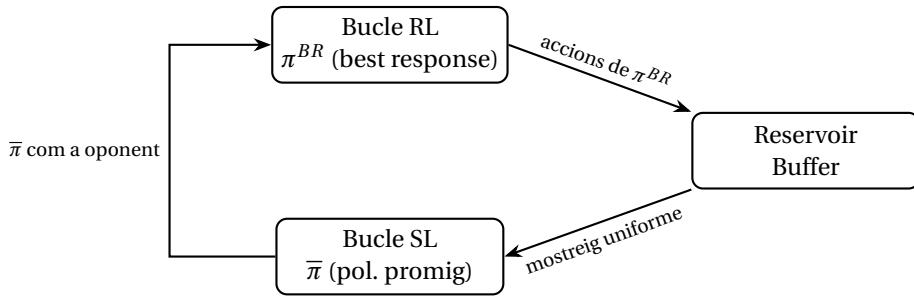


Figura 2.6: NFSP: el bucle RL genera accions que s’acumulen al reservoir buffer; el bucle SL aprèn per imitació la política promig $\bar{\pi}$, que s’usa com a oponent durant l’entrenament.

Reservoir Buffer

Per garantir que l’estimació de la política promig sigui imparcial, les accions del bucle RL s’emmagatzemen en un *Reservoir Buffer* que aplica mostreig uniforme de l’historial: cada acció passada té la mateixa probabilitat de romandre al buffer, independentment del moment en què es va generar.

Paràmetre η i Nash gap

El paràmetre η controla la probabilitat que l’oponent jugui la política promig (SL) en lloc del pool de self-play durant l’entrenament. La distància a l’equilibri de Nash es mesura com:

$$\text{exploit_vs_sl} = |WR_{\text{PPO vs SL}} - 50|.$$

Un valor de 0 indica que el best-response i la política promig estan en equilibri: cap dels dos pot guanyar sistemàticament a l’altre.

Taula 2.2: Comparativa dels algorismes d’AR emprats al treball.

Algorisme	On/Off-policy	Memòria	Garantia Nash
DQN	Off-policy	Experience replay	No
PPO	On-policy	Cap	No
NFSP	Tots dos	Reservoir buffer	Aproximada

EL JOC DEL TRUC

Abans d'entrenar cap agent cal disposar d'una implementació fidel i verificable del joc. Aquest capítol presenta primer les regles del Truc en la variant considerada, i tot seguit l'arquitectura de programari que les materialitza: un disseny Model-Vista-Controlador (MVC) que separa la lògica del joc, el control del flux i la interfície amb l'usuari. Aquesta separació és la que permet, més endavant, connectar el mateix motor de joc tant a una interfície humana com als entorns d'AR (capítol ??).

3.1 Regles del joc

El Truc és un joc de bases i apostes que es juga per parelles (dos contra dos) o un contra un. L'objectiu és ser el primer equip a assolir els 24 punts d'una *cama*. Es fa servir la baralla espanyola reduïda a 36 cartes: dels quatre colls —Ors, Copes, Espases i Bastos— s'eliminen els 2, 8 i 9, de manera que cada coll conserva nou rangs: {1, 3, 4, 5, 6, 7, 10, 11, 12}.

Jerarquia de les cartes

A diferència d'altres jocs, el valor de combat de les cartes no segueix l'ordre numèric. La taula 3.1 recull la jerarquia completa tal com la implementa el jutge del joc (TrucJutger): les quatre cartes superiors són úniques i reben noms propis, mentre que la resta s'ordena per rang amb desempats entre colls.

Cada mà es reparteix amb tres cartes per jugador i es disputa en dues apostes independents que es resolen per separat: **l'Envit** i **el Truc**.

L'Envit

L'Envit és l'aposta sobre la força de la mà en punts. El valor d'una mà es calcula sumant els valors de les dues cartes més altes d'un mateix coll i afegint-hi 20 punts base. A efectes d'aquesta suma, les figures (10, 11 i 12) valen 0, amb dues excepcions: el

3. EL JOC DEL TRUC

Posició	Carta	Denominació
1	11 de Bastos	L'Amo
2	10 d'Ors	Sa Madona
3	As d'Espases	L'espasa
4	As de Bastos	El basto
5	7 d'Espases	Manilla
6	7 d'Ors	Manilla
7	Els tresos	—
8	Assos d'Ors i de Copes	Assos bords
9	Els dotzes	Figures
10	Els onzes	Figures
11	Els deus	Figures
12	7 de Bastos i de Copes	Sets bords
13	Els sisos	
14	Els cincs	Blanques
15	Els quattres	

Taula 3.1: Jerarquia de força de les cartes, de major a menor.

10 d'Ors compta 7 i l'11 de Bastos compta 8, i tots dos lliguen amb qualsevol coll. Així, una mà amb el 7 i el 6 d'Espases val $7 + 6 + 20 = 33$ punts.

Quan un jugador canta envit, l'equip contrari pot acceptar, apujar l'aposta o retirar-se cedint un punt. Cada crit incrementa els punts en joc segons l'escala de la taula 3.2.

Crit	Punts en joc
Envit	2
Un altre	4
Dos més	6
La Falta	els punts que falten al líder per completar la cama

Taula 3.2: Escala d'apostes de l'Envit.

El Truc

El Truc és l'aposta sobre qui guanya les cartes. La mà es resol en fins a tres rondes (anomenades *bassetges*): a cada ronda els jugadors juguen una carta i en guanya la de rang superior segons la taula 3.1; l'equip que s'endú dues bassetges guanya la mà. En qualsevol moment un jugador pot cantar *Truc*, obrint una escala d'apostes independent de l'Envit (taula 3.3). Com en l'Envit, el contrari pot acceptar, contra-apostar o retirar-se cedint el nivell anterior.

Les senyes

En la modalitat per parelles, els companys es comuniquen mitjançant *senyes*: un conjunt de nou gestos facials codificats que indiquen quines cartes fortes es tenen,

Crit	Punts en joc
Truc	3
Retruc	6
Quatre Val	9
Joc Fora	tota la cama

Taula 3.3: Escala d'apostes del Truc.

sense parlar i procurant que els rivals no els vegin. Per exemple, alçar les celles indica que es té L'Amo, i aclucar un ull, Sa Madona. La implementació recull aquests nou senyals com a accions del joc, però el seu ús queda restringit a les partides 2 contra 2 entre humans; l'entrenament dels agents es fa sobre la variant 1 contra 1 sense senyals (vegeu el capítol ??).

3.2 Arquitectura del simulador

El simulador segueix el patró Model-Vista-Controlador (MVC), que separa el sistema en tres responsabilitats independents:

- El **model** conté la lògica del joc: l'estat, les regles i els jugadors automàtics.
- La **vista** gestiona l'entrada i la sortida amb l'usuari.
- El **controlador** orquestra el flux, demanant dades i accions a la vista i consultant i modificant l'estat a través del model.

La clau del disseny és que el controlador no depèn de cap implementació concreta, sinó de dues *interfícies* (contractes): qualsevol vista i qualsevol model que les satisfacin són intercanviables sense tocar el controlador. Aquesta indirecció és la que permet reutilitzar el mateix nucli de joc per a una interfície gràfica humana o per als adaptadors d'AR. La figura 3.1 resumeix aquesta organització.

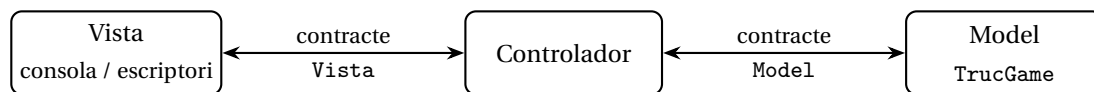


Figura 3.1: Arquitectura MVC: el controlador només coneix les interfícies.

Els contractes

El controlador parla amb el model a través del protocol Model (`interficie_model.py`) i amb la vista a través del protocol Vista (`interficie_vista.py`). Les taules 3.4 i 3.5 en resumeixen els mètodes principals.

Mètode	Funció
<code>iniciar(config)</code>	Crea i inicialitza una partida
<code>get_estat(pid)</code>	Retorna l'estat visible per a un jugador
<code>get_jugador_actual</code>	Indica qui ha de jugar ara
<code>es_huma(pid)</code>	Indica si el jugador és humà
<code>get_accio_bot(pid)</code>	Retorna l'acció triada per un bot
<code>aplicar_accio(a)</code>	Aplica una acció i avança l'estat
<code>es_final()</code>	Indica si la partida ha acabat
<code>get_resultat()</code>	Retorna el marcador i els <i>payoffs</i> finals

Taula 3.4: Mètodes principals del contracte Model.

Mètode	Funció
<code>demanar_config()</code>	Demana la configuració de la partida
<code>mostrar_estat(estat)</code>	Mostra l'estat actual del joc
<code>escollir_accio(legals)</code>	Presenta les accions legals i en retorna una
<code>mostrar_accio(pid, nom)</code>	Informa de l'acció feta per un jugador
<code>mostrar_fi_partida(...)</code>	Mostra el resultat final
<code>demanar_repetir()</code>	Pregunta si es vol jugar una altra partida

Taula 3.5: Mètodes principals del contracte Vista.

El bucle de joc

El controlador (`Controlador.executar_partida`) implementa un bucle únic que avança mentre la partida no acabi. A cada iteració consulta qui té el torn; si és un humà, mostra l'estat per la vista, li demana una acció i l'aplica al model; si és un bot, obté l'acció directament del model i l'aplica. Després de cada jugada comunica a la vista els esdeveniments rellevants —qui ha guanyat l'envit o el truc— i, en acabar, mostra el resultat. Tota la lògica de joc resta al model i tota la presentació resta a la vista; el controlador només coordina.

L'adaptador principal del model és `ModelInteractiu`, que embolcalla una instància de `TrucGame` i li afegeix dues capacitats: aïllar el motor de joc darrere del contracte i injectar models d'AR preentrenats als jugadors automàtics, de manera que un bot pugui ser tant un agent aleatori com una xarxa neuronal entrenada.

Les interfícies

El projecte ofereix dues vistes plenament funcionals. La **vista de consola** és una interfície de text, lleugera i adequada per a execucions automàtiques i depuració en entorns sense pantalla. La **vista d'escriptori** és una interfície gràfica que representa l'estat amb imatges reals de naips i recull les accions per clic de ratolí, intercalant retards artificials en els moviments dels bots per emular el ritme d'un rival humà. La interfície gràfica permet jugar partides 1 contra 1 —humà contra humà o humà contra agent— i 2 contra 2 entre humans amb suport de senyes.

3.3 El motor de joc

El cor del model és la classe `TrucGame`, que gestiona tot l'estat de la partida —jugadors, cartes, apostes, rondes i puntuació— de manera independent de qualsevol interfície. Delega responsabilitats concretes en tres rols especialitzats: `TrucDealer` barreja i reparteix les cartes; `TrucJudger` resol la jerarquia, els guanyadors de rondes i el càlcul de l'envit; i `TrucPlayer` representa cada jugador, mantenint la mà actual, la mà inicial —necessària per resoldre l'envit encara que les cartes ja s'hagin jugat— i, opcionalment, el model que decideix les seves accions.

Accions i estat de resposta

El conjunt d'accions del joc és fix i consta de 19 accions: tres per jugar cada carta de la mà, set per a les apostes i respostes (apostar envit, apostar truc, acceptar, retirar-se i passar) i nou per a les senyes. En cada torn, el motor restringeix les accions legals segons la situació. Un mecanisme central per a aquesta restricció és l'*estat de resposta*, que pot prendre tres valors: sense aposta pendent, truc pendent o envit pendent. Mentre hi ha una aposta pendent de resposta, l'adversari només pot acceptar-la, contra-apostar-la o retirar-se: no pot jugar cartes ni fer cap altra acció fins que l'aposta es resolgui.

Les fases del torn

Per eficiència en l'entrenament, l'estat d'un torn s'ha reduït a dues fases (figura 3.2). La **Fase 0** (senyals) només existeix quan les senyes estan actives i permet registrar un gest abans de jugar; si estan desactivades, se salta completament. La **Fase 1** unifica apostes i joc de cartes: és on es canta l'envit o el truc, es responen les apostes i es juguen les cartes.

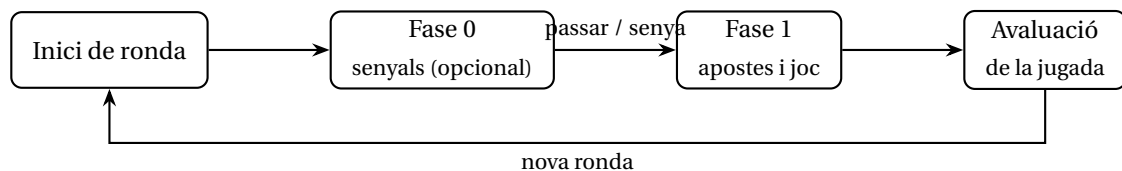


Figura 3.2: Fases d'un torn. Si les senyes estan desactivades, la Fase 0 se salta.

Fi de mà i fi de partida

El motor distingeix dos nivells de finalització. La funció `is_ma_over` indica que la mà actual ha acabat —perquè hi ha un guanyador per rondes o perquè algú s'ha retirat—, moment en què es reparteixen els punts i es reparteix una mà nova. La funció `is_over` indica que la partida sencera ha acabat, quan un equip arriba a la puntuació final de la cama.

A banda del resultat final de la partida, el motor emet també senyals de recompensa intermèdia (`reward_intermedis`) després de cada jugada rellevant. Aquestes senyals són la base del *reward shaping* que guia l'aprenentatge dels agents; el seu disseny

3. EL JOC DEL TRUC

detallat es presenta en el context dels entorns d'AR (capítol ??) i de la metodologia d'entrenament (capítol ??).

CAPÍTOL 4

CONCLUSIONS

[Conclusions del treball: resum del context i del problema, principals conclusions, implicacions teòriques i pràctiques, limitacions i treball futur.]



HIPERPARÀMETRES DELS MODELS

[Taules amb els hiperparàmetres complets de cada algorisme (DQN-RLCard, NFSP-RLCard, DQN-SB3, PPO-SB3) per a cada fase experimental.]

APÈNDIX

RESULTATS COMPLETS PER FASE

[Taules amb tots els valors numèrics dels experiments: mètriques finals, temps d'entrenament, etc.]

BIBLIOGRAFIA

- [1] J. Huizinga, *Homo Ludens: A Study of the Play-Element in Culture*. Routledge & Kegan Paul, 1938. 1
- [2] M. Campbell, A. J. Hoane Jr., and F.-h. Hsu, “Deep Blue,” *Artificial Intelligence*, vol. 134, no. 1–2, pp. 57–83, 2002. 1.1
- [3] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, pp. 484–489, 2016. 1.1
- [4] C. Berner *et al.*, “Dota 2 with large scale deep reinforcement learning,” 2019, arXiv:1912.06680. 1.1
- [5] N. Brown and T. Sandholm, “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals,” *Science*, vol. 359, no. 6374, pp. 418–424, 2018. 1.2, 2.2
- [6] ———, “Superhuman AI for multiplayer poker,” *Science*, vol. 365, no. 6456, pp. 885–890, 2019. 1.2, 2.2, 2.7
- [7] Retruc.net, “Reglament del truc,” <https://retruc.net/reglamenttruc>, 2024, accedit: 2026. 1.3
- [8] D. Zha, K.-H. Lai, Y. Cao, S. Huang, R. Wei, J. Guo, and X. Hu, “RLCard: A toolkit for reinforcement learning in card games,” *arXiv preprint arXiv:1910.04376*, 2019. 1
- [9] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. 1
- [10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. 2.1
- [11] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015. 2.3
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. 2.4, 2.4
- [13] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, 2009, pp. 41–48. 2.5

- [14] M. Hausknecht and P. Stone, “Deep recurrent Q-learning for partially observable MDPs,” *arXiv preprint arXiv:1507.06527*, 2015. 2.6
- [15] J. Heinrich and D. Silver, “Deep reinforcement learning from self-play in imperfect-information games,” *arXiv preprint arXiv:1603.01121*, 2016. 2.8